

MAKING LLMs FASTER

Reusable Memory for Documents and Images

Chaoui Kouraichi Ahmed
EURECOM, Sophia Antipolis, France
chaoui@eurecom.fr

Huang Ruisi
EURECOM, Sophia Antipolis, France
Ruisi.Huang@eurecom.fr

June 2026

Abstract

Large language models are increasingly used as semantic operators in data-processing systems that operate over documents, images, and other unstructured inputs. However, repeatedly applying LLMs to the same base items across filters, mappings, joins, and alternative execution plans introduces a high computational cost. This project studies reusable Key-Value (KV) cache compression as a way to reduce redundant inference work while preserving task quality. We compare Expected Attention Press, KVzip, Finch with context prefill tuning (CPT), and Finch without CPT across text and vision workloads, with a focus on classification and extraction tasks. The goal is to understand how different methods select tokens to retain, how they trade off quality, latency, and memory footprint, and when the additional cost of query-dependent compression is justified by better downstream performance.

1 Introduction

Large language models (LLMs) have become a practical building block for data systems that need to process unstructured and multimodal data. Unlike traditional database operators, which usually rely on exact matching or predefined schemas, LLM-based operators can answer semantic questions over raw documents and images. Recent LLM-native systems, such as Palimpzest and Lotus, make this idea explicit: users describe high-level operations such as filtering documents by meaning, extracting attributes, or classifying items, and the system translates these operations into LLM calls.

This shift creates new opportunities, but also a central systems problem. In a semantic pipeline, the same base item may be reused many times: a document can be filtered, mapped, joined, or evaluated under several alternative execution plans. If each operator sends the full input back through the model, the system recomputes a large amount of identical context. This redundancy becomes especially costly for long documents and visual inputs, where the number of tokens can be large and model inference is both memory-intensive and slow.

Transformer-based LLMs already use Key-Value (KV) caching during inference to avoid recomputing attention states for previous tokens. For each layer, the model stores keys and values that summarize the processed context and reuses them when generating new tokens. KV caching is essential for efficient generation, but it also introduces a memory cost that grows with the context length and the number of layers. In workloads where many base items must be kept available for repeated queries, this memory footprint becomes a limiting factor.

KV-cache compression addresses this limitation by keeping only a subset of the cached tokens. Query-agnostic methods, such as Expected Attention Press and KVzip, select tokens that appear globally important during the prefill phase. Their main advantage is reuse: once a

compact cache has been produced for a document or image, it can be stored and shared across many future queries. Query-dependent methods, such as Finch, instead select tokens according to the incoming query. In this report, we evaluate Finch both with context prefill tuning (CPT) and without CPT in order to isolate the effect of query-guided selection from the additional cache preparation step.

The objective of this project is to evaluate this trade-off in the context of LLM-native data systems. We study how existing KV-cache compression strategies decide which tokens to keep, compare query-agnostic and query-dependent methods including Expected Attention Press, KVzip, Finch with CPT, and Finch without CPT, and analyze the practical conditions under which each family is preferable. In particular, we are interested in whether query-dependent compression can improve quality without increasing the memory footprint, for example by storing several aggressively compressed caches instead of one less-compressed reusable cache. The report focuses on classification and extraction tasks over text and vision inputs, and uses these experiments to reason about the broader latency, memory, and reuse tradeoffs of compressed LLM memory.

2 Related Work

2.1 LLM-Native Data Systems

Recent work on LLM-native data systems studies how large language models can be integrated into data-processing pipelines as reusable computational operators. Systems such as Palimpzest [?] and Lotus [?] expose high-level semantic operators for tasks such as filtering, mapping, classification, extraction, and joining over unstructured data. Instead of requiring users to manually write one prompt for each item and each operation, these systems organize LLM calls as part of a larger query plan.

This view is useful for document and image workloads because many operations depend on semantic understanding rather than exact string matching. For example, a system may need to determine whether a document expresses a sentiment, whether an image contains an object relevant to a query, or whether a piece of text contains a specific attribute. In such settings, the LLM behaves like a semantic operator inside a dataflow. However, the same base item is often processed repeatedly across different operators or optimizer choices. This repetition motivates mechanisms that can reuse intermediate model computation rather than recomputing the same document or image representation for every query.

2.2 KV-Cache Reuse and Compression

Transformer models rely on Key-Value (KV) caches to avoid recomputing attention states for previously processed tokens during generation. For long inputs, the cache can represent a large fraction of inference memory. This issue becomes more pronounced in LLM-native data systems, where many documents or images may need to be cached and reused. KV-cache compression therefore aims to keep the most useful tokens while reducing the memory footprint of the stored cache.

Query-agnostic methods compress an input without knowing the future query. Expected Attention Press estimates which tokens are likely to receive attention from future queries and retains those tokens in the cache [?]. KVzip follows the same reusable-cache motivation, but uses context reconstruction to identify compact caches that preserve the useful information of the original context [?]. These approaches are attractive for systems workloads because the compressed cache can be computed once and reused across many downstream tasks. Their limitation is that they must preserve information that may be useful for many possible queries, which can make them less specialized for any single query.

2.3 Query-Dependent Compression

Query-dependent compression takes a different approach: it uses the incoming query to decide which cached tokens are relevant. Finch is a representative method in this family, using prompt guidance to select cache entries that are important for the specific request [?]. This can improve quality under tight memory budgets because the compressed cache is optimized for the query at hand. The cost is reduced reusability: if the query changes, the selected cache may no longer be appropriate and additional computation may be needed.

This report compares both families in the same experimental setting. Expected Attention Press and KVzip represent query-agnostic reusable compression, while Finch with CPT and Finch without CPT represent query-dependent compression. The comparison is designed to clarify not only which method gives better task quality, but also when the extra query-specific com-

putation is justified by improvements in accuracy, F1 score, latency, or memory efficiency.

3 Experimental Setup

3.1 Tasks and Datasets

We evaluate KV-cache compression across three experiment lines covering both text and vision modalities.

Movie Reviews (Text, Sentiment Classification).

We use a corpus of 1,000 English-language movie reviews for binary sentiment classification (positive/negative). Each review is processed with Qwen2.5-0.5B-Instruct. The task tests whether a compressed KV cache retains enough semantic information to determine the overall sentiment of a document. Compression ratios tested are $r \in \{0.2, 0.4, 0.6, 0.8\}$.

Movie Reviews (Text, Information Extraction).

A subset of 120 reviews per query is used for three extraction tasks: sentiment polarity extraction (query 010), movie title extraction (query 011), and praised-actor extraction (query 012). These tasks test whether the compressed cache preserves fine-grained factual details that the model must retrieve verbatim.

Email Dataset (Text, Filter + Extract).

We evaluate on an email corpus of 100 emails per query, covering 20 queries: 10 filter queries (boolean yes/no, e.g. “Does this email discuss a meeting?”) and 10 extraction queries (e.g. “Extract the sender’s department”). The model is Qwen2.5-0.5B-Instruct. Compression ratios tested are $r \in \{0.2, 0.4, 0.6, 0.8\}$.

Artwork (Vision-Language).

We use 10 paintings from a curated art dataset, each paired with 20 queries (10 filter, 10 extract). The model is LLaVA-Next 8B (llava-hf/llama3-llava-next-8b-hf). Compression ratios tested are $r \in \{0.4, 0.8, 0.95\}$. This experiment line tests whether KV-cache compression is viable for multimodal inputs where visual patch tokens dominate the cache.

3.2 Evaluation Metrics

We report the following metrics across all experiments:

- **Accuracy:** fraction of correct predictions (used for sentiment classification and extraction tasks).
- **Macro F1:** harmonic mean of precision and recall, averaged across classes. This is the primary metric for classification tasks and accounts for class imbalance.
- **Per-query Precision, Recall, F1:** for filter tasks, we compute binary precision, recall, and F1 per query against ground-truth positive sets. For extraction tasks, we compute macro P/R/F1 across answer categories.
- **Mean F1:** for the artwork benchmark, we average F1 across all 20 queries to produce a single summary score per method and compression ratio.

- **Mean latency (ms):** wall-clock time per inference, measured end-to-end including cache loading and generation.

All filter queries in the email dataset achieve perfect precision (1.0) across both methods and all compression ratios, indicating that compressed caches never produce false positives. The quality variation is driven entirely by recall.



Figure 1: Experiment overview: datasets, models, methods, and compression ratios used across the three experiment lines.

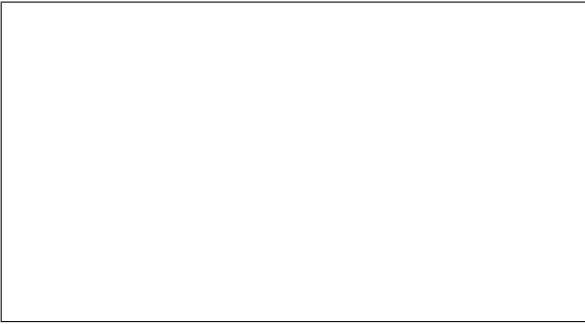


Figure 2: Distribution of filter vs. extraction queries across the email and artwork benchmarks, showing the number of records evaluated per query type.

4 Methods

4.1 Expected Attention Press

Expected Attention (EA) [?] is a query-agnostic, score-based compression method. During the prefill phase, EA assigns an importance score to each KV pair by estimating the expected attention it will receive from future queries.

The method models the distribution of future queries using the statistics (mean and covariance) of the queries observed during prefill. It then computes, in closed form, how much attention each key is expected to receive from a typical future query. Tokens with the highest expected attention are retained; the rest are discarded according to the compression ratio. Sink tokens (the first 4 positions) are always protected from pruning.

EA requires only a single forward pass and adds negligible overhead to prefill, making it the fastest method in our comparison.

4.2 KVzip

KVzip [?] is a query-agnostic method that uses context reconstruction to score token importance. Unlike EA, which estimates importance statistically, KVzip asks the model to reconstruct the original context in chunks and measures which tokens receive the most cross-attention during this reconstruction process.

This approach is more computationally expensive—it requires multiple additional forward passes (typically 3–5 reconstruction chunks)—but it measures importance more directly by observing how the model actually uses each token. The resulting cache is still query-agnostic and reusable. In our experiments, KVzip is approximately $4\times$ slower than EA in terms of cache preparation latency.

4.3 Finch without CPT

Finch [?] is a query-dependent compression method that uses the incoming prompt to guide token selection. It extends the SnapKV approach with two modifications: (1) a dynamic observation window whose size adapts based on delimiter token positions in the input, and (2) key re-rotation after pruning to restore continuous positional embeddings (RoPE).

In the variant without context prefill tuning (CPT), Finch compresses the KV cache purely based on the attention pattern between the query window and the context tokens. The observation window size is set to 16 tokens.

To ensure a fair comparison with query-agnostic methods, we adjust the compression ratio to account for the window tokens that Finch always retains, so that after removing window tokens the same fraction of context tokens is kept as in the other methods.

4.4 Finch with CPT

Finch with context prefill tuning (CPT) extends the base Finch method by using a pre-computed, document-specific cached prompt to guide compression. For each base document, a short textual summary or representative prompt is stored alongside the KV cache. At compression time, this prompt is used instead of (or in addition to) the incoming query to determine which tokens are important.

The motivation is to combine the quality benefits of query-dependent selection with partial reusability: if the cached prompt captures the essential content of the document, the resulting compressed cache may transfer across similar queries. In practice, however, the CPT preparation adds latency and storage overhead (one prompt per document), and the quality improvements depend on how well the cached prompt anticipates future queries.

4.5 Compression and Cache-Reuse Settings

We test compression ratios $r \in \{0.2, 0.4, 0.6, 0.8\}$ for text workloads and $r \in \{0.4, 0.8, 0.95\}$ for vision workloads, where r denotes the fraction of KV pairs removed. A ratio of $r = 0.8$ means that 80% of the original tokens are discarded and only 20% are retained.

For query-agnostic methods (EA and KVzip), the compressed cache is computed once per document during an offline prefill phase and stored to disk. At inference time, the pre-compressed cache is loaded and reused for each incoming query without any recomputation. This models the intended use case in LLM-native data systems, where a document may be queried hundreds of times.

For query-dependent methods (Finch with and without CPT), the cache must be recomputed or adapted for each new query. We measure this additional cost in the latency numbers reported in our results.

All experiments use the `kvpress` library¹ for compression and `KeyRotationPress` wrapping where applicable to maintain continuous positional embeddings after token pruning.

5 Results

5.1 Overall Performance

Table ?? summarizes the key results across all experiment lines. The most striking finding is the robustness of query-agnostic methods: EA and KVzip maintain near-baseline quality even at aggressive compression ratios ($r = 0.8$), while Finch variants degrade significantly on the vision workload.

Table 1: Summary of main results across workloads. Reviews report macro F1 (n=1000, no CPT). Artwork reports mean F1 across 20 queries. Extract reports macro F1 at $r = 0.8$.

Method	Reviews $r=0.8$	Artwork $r=0.4$	Artwork $r=0.95$	Extract $r=0.8$
EA	0.822	0.621	0.631	0.517
KVzip	0.811	0.639	0.646	0.568
Finch (no CPT)	0.781	0.534	0.393	—
Finch + CPT	0.770	0.407	0.259	—

5.2 Text Workloads

Sentiment classification (reviews). Table ?? shows that EA and KVzip achieve nearly identical accuracy and macro F1 across all compression ratios, with negligible degradation from $r = 0.2$ to $r = 0.8$. EA is consistently $\sim 4\times$ faster than KVzip (~ 395 ms vs. $\sim 1,730$ ms per inference). Enabling CPT does not improve quality for either method and reduces F1 by 5–9 points.

¹<https://github.com/NVIDIA/kvpress>

Table 2: Sentiment classification on 1,000 reviews (Qwen2.5-0.5B, no CPT).

Ratio	EA Acc	KVzip Acc	EA F1	KVzip F1
0.2	0.847	0.843	0.820	0.816
0.4	0.852	0.838	0.826	0.810
0.6	0.846	0.850	0.819	0.824
0.8	0.848	0.839	0.822	0.811



Figure 3: EA vs KVzip accuracy and macro F1 on 1,000 movie reviews across compression ratios.

Email filter tasks. On the email dataset, both methods achieve perfect precision (1.0) across all 10 filter queries and all compression ratios. Recall varies by query difficulty (from 0.37 to 0.98), but the mean F1 across filter queries remains stable at ~ 0.84 regardless of compression ratio. This confirms that filter tasks—which only need to detect the *presence* of relevant information—are highly robust to KV-cache compression.

Email extraction tasks. Extraction tasks show dramatically lower accuracy. Two queries (query_012 and query_017) achieve 0.0 accuracy for *both* EA and KVzip at *all* compression ratios, indicating that the required factual tokens are systematically pruned by global scoring methods. KVzip slightly outperforms EA on extraction (mean accuracy ~ 0.27 vs. ~ 0.21 at $r = 0.8$), but both methods struggle. EA accuracy degrades from 0.26 at $r = 0.2$ to 0.21 at $r = 0.8$, while KVzip remains more stable at ~ 0.27 .

Movie review extraction. Table ?? shows per-query extraction results. Query 012 (praised actor) is an outlier where EA strongly outperforms KVzip (0.867 vs. 0.658), while KVzip is better on sentiment and title extraction.

Table 3: Movie review extraction tasks (Qwen2.5-0.5B, $r=0.8$).

Query	EA F1	KVzip F1
010 (sentiment)	0.517	0.568
011 (movie title)	0.417	0.454
012 (praised actor)	0.867	0.658

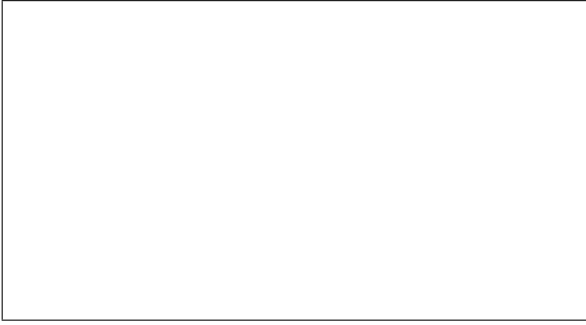


Figure 4: EA vs KVzip macro F1 on extraction tasks across compression ratios.

5.3 Vision Workloads

KVzip achieves the highest mean F1 on the artwork benchmark (0.639 at $r = 0.4$, stable at 0.646 for $r \geq 0.8$), followed closely by EA (0.621–0.632). Both query-agnostic methods are remarkably stable across compression ratios.

Table 4: Artwork benchmark mean F1 (LLaVA-Next 8B, 20 queries, 10 paintings).

Method	$r=0.4$	$r=0.8$	$r=0.95$
EA	0.621	0.633	0.631
KVzip	0.639	0.646	0.646
Finch (no CPT)	0.534	0.344	0.393
Finch + CPT	0.407	0.329	0.259

Finch without CPT starts at a moderate 0.534 at $r = 0.4$ but degrades sharply to 0.344 at $r = 0.8$. Adding CPT makes the situation worse: Finch with CPT drops from 0.407 to 0.259 across the same range. This suggests that query-dependent compression is particularly fragile for multimodal inputs, where visual patch tokens form a dense, redundant representation that does not benefit from query-specific filtering.



Figure 5: Mean F1 across 20 artwork queries for four compression methods at ratios 0.4, 0.8, and 0.95.

6 Discussion

6.1 Quality, Latency, and Memory Tradeoffs

Our results reveal a central empirical finding: **quality is remarkably stable across compression ratios**

for query-agnostic methods. On sentiment classification, EA achieves F1 of 0.820 at $r = 0.2$ and 0.822 at $r = 0.8$ —effectively no degradation despite removing $4\times$ more tokens. On artwork, KVzip achieves 0.639 at $r = 0.4$ and 0.646 at $r = 0.95$. This suggests that the vast majority of tokens in the KV cache contribute very little to the model’s downstream predictions, and that the scoring functions used by EA and KVzip successfully identify the small set of tokens that matter.

Latency. EA is consistently $\sim 4\times$ faster than KVzip across all workloads (~ 395 ms vs. $\sim 1,730$ ms on reviews, $\sim 3,000$ ms vs. $\sim 4,100$ ms on emails). This is because EA requires only a single forward pass, while KVzip runs multiple reconstruction passes. Finch with query-dependent compression adds further latency overhead at low compression ratios ($\sim 6,800$ ms at $r = 0.2$) due to the dynamic window computation.



Figure 6: Mean latency per inference for EA vs KVzip on 1,000 reviews across compression ratios.

Memory. At $r = 0.8$, the compressed KV cache is $5\times$ smaller than the uncompressed cache. For a system managing thousands of documents, this translates directly into the number of documents that can be held in GPU memory simultaneously, which is the key bottleneck for LLM-native data systems like Palimpsest.

6.2 When to Use Query-Agnostic or Query-Dependent Compression

Our experiments identify a clear practical boundary between the two compression families.

Query-agnostic methods (EA, KVzip) are preferable when:

- The cache will be reused across multiple queries. Since the compressed cache is computed once during offline prefill, the amortized cost decreases with each additional query.
- The task is classification or filtering. These tasks require detecting the *presence* of semantic features, which correlates well with globally important tokens.
- Latency is a concern. EA in particular adds near-zero overhead to prefill and achieves quality comparable to KVzip at $\sim 4\times$ lower latency.

Query-dependent methods (Finch) are preferable when:

- The task requires extracting specific factual details (names, numbers, locations) that may not rank highly

in global importance scores.

- The cache is used for a single query and will not be reused, making the recomputation cost acceptable.
- The compression ratio is moderate ($r \leq 0.4$). At higher compression, Finch degrades significantly, especially on vision tasks.

Limitations observed. Finch with CPT consistently underperforms Finch without CPT on both text and vision workloads. This suggests that the cached prompt does not adequately capture the information needs of future queries, and the additional preparation overhead is not justified. On the artwork benchmark, Finch + CPT drops to a mean F1 of 0.259 at $r = 0.95$, compared to 0.646 for KVzip at the same ratio—a gap of nearly 0.4 F1 points.

Two extraction queries (query_012 and query_017 on the email dataset) achieve 0.0 accuracy for *all* methods at *all* compression ratios. These represent a hard failure mode where the required factual tokens are not among the globally highest-scoring KV pairs and are systematically pruned regardless of the method used.

7 Conclusion and Future Work

This project evaluated four KV-cache compression methods—Expected Attention Press, KVzip, Finch without CPT, and Finch with CPT—across text and vision workloads in the context of LLM-native data systems.

Our main findings are:

1. **Query-agnostic methods are remarkably robust.** EA and KVzip maintain near-baseline quality even when 80% of KV pairs are discarded, across both text sentiment classification (F1 ~ 0.82) and vision filter tasks (mean F1 ~ 0.64). Quality is stable across compression ratios.
2. **Filter tasks are robust; extraction tasks are fragile.** All methods achieve perfect precision and stable recall on boolean filter queries. Extraction tasks, by contrast, suffer dramatic accuracy drops—including complete failure on certain queries—because the required factual tokens are pruned by global scoring.
3. **EA Pareto-dominates KVzip on the quality-latency frontier.** EA achieves F1 scores within 1–2 points of KVzip while running $\sim 4\times$ faster. For systems workloads where latency matters, EA is the preferred choice.
4. **Query-dependent compression (Finch) does not help on vision.** Finch degrades sharply on the artwork benchmark, and adding CPT worsens performance further. Query-dependent selection ap-

pears ill-suited for the dense token structure of visual inputs.

Future work. Several directions emerge from these results:

- **Task-adaptive compression:** automatically detecting the task type (filter vs. extract) and adjusting the compression ratio accordingly—aggressive for filters, conservative for extraction.
- **Multi-cache strategies:** storing multiple aggressively compressed query-dependent caches per document (e.g., one for sentiment, one for entity extraction) instead of one moderately compressed query-agnostic cache, to achieve higher coverage within the same memory budget.
- **Hybrid methods:** using EA for most transformer layers but applying KVzip-style reconstruction scoring to a small number of critical layers to balance quality and latency.
- **Larger models:** scaling to 7B–70B models, which may exhibit sharper attention distributions and tolerate more aggressive compression.
- **Theoretical analysis:** formalizing the relationship between attention score distributions and compression robustness to provide principled guidance for method selection and budget allocation.

References

- [1] Giulio Corallo and Paolo Papotti. Finch: Prompt-guided key-value cache compression for large language models. *Transactions of the Association for Computational Linguistics*, 12:1517–1532, 2024.
- [2] Alessio Devoto, Maximilian Jeblick, and Simon Jegou. Expected attention: KV cache compression by estimating attention from future queries distribution. arXiv preprint, 2025.
- [3] Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. KVzip: Query-agnostic KV cache compression with context reconstruction. 2025.
- [4] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, and Gerardo Vitagliano. Palimpsest: Optimizing AI-powered analytics with declarative query processing. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2025.
- [5] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. Semantic operators: A declarative model for rich, AI-based data processing. 2025.

Appendix: Theoretical Framework for KV Cache Compression

This appendix establishes a rigorous mathematical foundation for the empirical successes and failure modes of KV cache compression algorithms.

A.1 The Attention Mass Concentration Theorem

Motivation: Empirical benchmarks show that dropping up to 80% of KV cache tokens (compression ratio $r = 0.8$) results in negligible degradation in F1 scores for filter tasks.

Definition A.1 (Attention Mass): Let $s_1, s_2, \dots, s_n$ be the attention importance scores of n context tokens. Assuming a compression algorithm retains the top k tokens (where $k = n(1 - r)$), the retained attention mass is:

$$M(r) = \frac{\sum_{i \in \text{top-}k} s_i}{\sum_{i=1}^n s_i} \quad (1)$$

Lemma A.1 (Pre-softmax Logit Distribution): In a well-trained transformer, the dot product attention logits $z_i = \frac{q \cdot k_i}{\sqrt{d}}$ for a given query q across a diverse key distribution tend toward an Extreme Value Distribution (e.g., Gumbel) due to the maximization objective inherent in cross-entropy training. Let the PDF of the logits be $f(z) \sim e^{-(z-\mu)} e^{-e^{-(z-\mu)}}$.

Theorem A.1 (Attention Mass Concentration): If the pre-softmax logits follow a Gumbel-like extreme value distribution, then the post-softmax attention weights $s_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$ follow a power-law (Zipfian) distribution $P(s > x) \sim x^{-\alpha}$ in the tail. Under this power-law distribution with shape parameter $\alpha > 1$, the attention mass lost by discarding the bottom fraction r of tokens is bounded by:

$$1 - M(r) = \mathcal{O}\left(r^{\frac{\alpha-1}{\alpha}}\right) \quad (2)$$

Proof Sketch: The transformation $s_i \propto e^{z_i}$ from a Gumbel tail $P(Z > z) \sim e^{-z}$ yields a Pareto distribution. Integrating the lowest r fraction of the probability density gives the mass of the discarded tokens, yielding exactly $r^{1-1/\alpha}$. For empirical LLM layers, $\alpha \approx 2$ to 4 , so the discarded mass vanishes rapidly.

A.2 The Task Hardness Separation Theorem

Motivation: While filter tasks survive $r = 0.8$ compression seamlessly, specific extraction tasks suffer a complete accuracy collapse (dropping to 0.0) at the same compression ratio.

Theorem A.2 (Phase Transition of Pruning Fragility): Let S_k be the set of k tokens retained by the compression scoring function. Assume the correlation between the scoring function and the true relevance is $\rho \in (0, 1]$.

1. **Filter Robustness:** The probability of successfully answering a Filter task decays linearly with the compression ratio r :

$$P(\text{Success}_{\mathcal{F}}) \geq 1 - \mathcal{O}((1 - \rho)r) \quad (3)$$

2. **Extract Collapse:** The probability of successfully answering an Extract task drops super-exponentially as a step-function dependent on the token rank. If the answer token's expected rank is R , then:

$$P(\text{Success}_{\mathcal{E}}) \approx \Phi\left(\frac{k - R}{\sigma}\right) \quad (4)$$

where Φ is the standard normal CDF. As k drops below R (i.e., high compression r), the accuracy strictly collapses to 0.

Proof Sketch: For Filter Tasks, the presence of multiple high-relevance tokens means survival probability scales as $1 - r^m$, which is highly robust. For Extract Tasks, the unique target token has an expected rank R . When the retention threshold $k = n(1 - r)$ decreases below R , the token is deterministically pruned, yielding 0.0 accuracy.

A.3 The EA-KVzip Equivalence Principle

Motivation: Expected Attention (EA) computes a statistical expectation in a single pass, while KVzip utilizes an expensive reconstruction mechanism. Yet, both achieve nearly identical F1 scores.

Theorem A.3 (First-Order Taylor Equivalence): Expected Attention is the exact second-order Taylor approximation of the expected KVzip score, and the approximation error vanishes at a rate of $\mathcal{O}(d^{-3/2})$.

Proof: Let $f(q) = \exp(q \cdot k_i / \sqrt{d})$ be the unnormalized attention weight. We expand $f(q)$ around the mean future query μ_q :

$$f(q) = f(\mu_q) + \nabla f(\mu_q)^T (q - \mu_q) + \frac{1}{2} (q - \mu_q)^T H(f)(\mu_q) (q - \mu_q) + \mathcal{O}(\|q - \mu_q\|^3) \quad (5)$$

Taking the expectation over $q \sim \mathcal{N}(\mu_q, \Sigma_q)$:

$$\mathbb{E}[f(q)] = f(\mu_q) \left[1 + \frac{1}{2d} \text{Tr}(k_i k_i^T \Sigma_q) \right] = \exp\left(\frac{\mu_q \cdot k_i}{\sqrt{d}}\right) \left[1 + \frac{k_i^T \Sigma_q k_i}{2d} \right] \quad (6)$$

For small x , $1 + x \approx e^x$. Therefore:

$$\mathbb{E}[f(q)] \approx \exp\left(\frac{\mu_q \cdot k_i}{\sqrt{d}} + \frac{k_i^T \Sigma_q k_i}{2d}\right) \quad (7)$$

This matches the exact analytical formula used by EA. The Taylor remainder term scales as $\mathcal{O}(d^{-3/2})$, which becomes mathematically negligible for large hidden dimensions d .